

Step 1: Initialization & Universe Definition

In this cell, we import the required libraries (primarily `scikit-fuzzy`) and define the exact numerical ranges (Universe of Discourse) for our 8 system inputs and the single output representing the purchase suitability score.

```
In [2]: !pip install numpy matplotlib scipy networkx scikit-fuzzy
```

```
Requirement already satisfied: numpy in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (2.4.4)
Requirement already satisfied: matplotlib in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (3.10.8)
Requirement already satisfied: scipy in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (1.17.1)
Requirement already satisfied: networkx in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (3.6.1)
Requirement already satisfied: scikit-fuzzy in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (0.5.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\data\hit\תמונה עיבוד\venv\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 23.2.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [3]: import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# 1. Define Input Variables (Antecedents) and Output (Consequent)
# The universe of discourse is defined using np.arange(start, stop, step).
# Step is 1 for integers, or smaller (e.g., 0.1) for precise decimals.

budget_dev = ctrl.Antecedent(np.arange(-20, 21, 1), 'Budget_Deviation')
maint_cost = ctrl.Antecedent(np.arange(0, 11, 1), 'Maintenance_Costs')
fuel_eff = ctrl.Antecedent(np.arange(5, 26, 1), 'Fuel_Efficiency')
mileage = ctrl.Antecedent(np.arange(0, 251, 1), 'Mileage')
history = ctrl.Antecedent(np.arange(1, 6, 1), 'Ownership_History')
comfort = ctrl.Antecedent(np.arange(0, 11, 1), 'Practicality_Comfort')
condition = ctrl.Antecedent(np.arange(0, 101, 1), 'Car_Condition')
safety = ctrl.Antecedent(np.arange(1, 6, 0.5), 'Safety_Rating')

suitability = ctrl.Consequent(np.arange(0, 1.01, 0.01), 'Purchase_Suitability')

print("Cell 1 Executed: Variables and Universes defined successfully.")
```

Cell 1 Executed: Variables and Universes defined successfully.

Step 2: Fuzzification & Membership Functions

Here we translate the numerical values into linguistic categories. To achieve a realistic and reliable system behavior, we utilized a combination of triangular functions for specific target values, trapezoidal functions for extreme edge cases, and Gaussian (bell) curves for variables representing wear and subjective feelings, where transitions are gradual and smooth.

```

In [4]: # --- 1. Budget Deviation ---
# Using trapezoidal (trapmf) for extreme edges to capture all values beyond a threshold,
# and triangular (trimf) for internal points where there's a specific "ideal" value.
budget_dev['Significantly_Under'] = fuzz.trapmf(budget_dev.universe, [-20, -20, -15, -10])
budget_dev['Slightly_Under'] = fuzz.trimf(budget_dev.universe, [-15, -7.5, 0])
budget_dev['On_Budget'] = fuzz.trimf(budget_dev.universe, [-5, 0, 5])
budget_dev['Slightly_Over'] = fuzz.trimf(budget_dev.universe, [0, 7.5, 15])
budget_dev['Significantly_Over'] = fuzz.trapmf(budget_dev.universe, [10, 15, 20, 20])

# --- 2. Maintenance Costs ---
maint_cost['Very_Cheap'] = fuzz.trapmf(maint_cost.universe, [0, 0, 2, 4])
maint_cost['Cheap'] = fuzz.trimf(maint_cost.universe, [2, 4, 6])
maint_cost['Average'] = fuzz.trimf(maint_cost.universe, [4, 6, 8])
maint_cost['Expensive'] = fuzz.trimf(maint_cost.universe, [6, 8, 10])
maint_cost['Very_Expensive'] = fuzz.trapmf(maint_cost.universe, [8, 9, 10, 10])

# --- 3. Fuel Efficiency ---
fuel_eff['Poor'] = fuzz.trapmf(fuel_eff.universe, [5, 5, 8, 10])
fuel_eff['Below_Average'] = fuzz.trimf(fuel_eff.universe, [8, 11, 14])
fuel_eff['Average'] = fuzz.trimf(fuel_eff.universe, [12, 14, 16])
fuel_eff['Good'] = fuzz.trimf(fuel_eff.universe, [14, 17, 20])
fuel_eff['Excellent'] = fuzz.trapmf(fuel_eff.universe, [18, 21, 25, 25])

# --- 4. Mileage ---
# Using Gaussian (gaussmf) because wear and tear is a continuous, smooth physical process.
# There are no sharp boundaries in engine degradation. Syntax: gaussmf(universe, mean, sigma).
mileage['Very_Low'] = fuzz.gaussmf(mileage.universe, 0, 15)
mileage['Low'] = fuzz.gaussmf(mileage.universe, 50, 15)
mileage['Average'] = fuzz.gaussmf(mileage.universe, 105, 20)
mileage['High'] = fuzz.gaussmf(mileage.universe, 160, 15)
mileage['Very_High'] = fuzz.gaussmf(mileage.universe, 250, 20)

# --- 5. Ownership History ---
history['First_Hand'] = fuzz.trapmf(history.universe, [1, 1, 1, 1.5])
history['Second_Hand'] = fuzz.trimf(history.universe, [1, 2, 3])
history['Third_Hand'] = fuzz.trimf(history.universe, [2, 3, 4])
history['Multiple'] = fuzz.trapmf(history.universe, [3.5, 4, 5, 5])

# --- 6. Practicality & Comfort ---
# Gaussian MF used here to model human subjective feelings, which transition smoothly.
comfort['Basic'] = fuzz.gaussmf(comfort.universe, 0, 2)
comfort['Adequate'] = fuzz.gaussmf(comfort.universe, 4, 1.5)
comfort['Comfortable'] = fuzz.gaussmf(comfort.universe, 7, 1.5)
comfort['Premium'] = fuzz.gaussmf(comfort.universe, 10, 1.5)

# --- 7. Car Condition ---
# Gaussian MF chosen because mechanical deterioration happens gradually over time.
condition['Poor'] = fuzz.gaussmf(condition.universe, 0, 15)
condition['Fair'] = fuzz.gaussmf(condition.universe, 45, 12)
condition['Good'] = fuzz.gaussmf(condition.universe, 75, 10)
condition['Excellent'] = fuzz.gaussmf(condition.universe, 100, 10)

# --- 8. Safety Rating ---

```

```

safety['Unsafe'] = fuzz.trapmf(safety.universe, [1, 1, 1.5, 2.5])
safety['Poor'] = fuzz.trimf(safety.universe, [1.5, 2.5, 3.5])
safety['Acceptable'] = fuzz.trimf(safety.universe, [2.5, 3.5, 4.5])
safety['Good'] = fuzz.trimf(safety.universe, [3.5, 4, 4.5])
safety['Excellent'] = fuzz.trapmf(safety.universe, [4.5, 4.8, 5, 5])

# --- Output: Purchase Suitability ---
# Standard triangular MFs are best for the output to ensure clear, predictable
# centroid defuzzification.
suitability['Definitely_Avoid'] = fuzz.trimf(suitability.universe, [0, 0, 0.2
5])
suitability['Poor_Choice'] = fuzz.trimf(suitability.universe, [0.15, 0.3, 0.4
5])
suitability['Uncertain'] = fuzz.trimf(suitability.universe, [0.35, 0.5, 0.65])
suitability['Good_Choice'] = fuzz.trimf(suitability.universe, [0.55, 0.7, 0.8
5])
suitability['Highly_Recommended'] = fuzz.trimf(suitability.universe, [0.75, 1,
1])

# =====
# Custom Visualisation (All Variables, No Legends, Text on Peaks)
# =====

# List of all our variables and their display names for the graphs
all_vars = [
    (budget_dev, 'Budget Deviation (%)'),
    (maint_cost, 'Maintenance Costs (Scale 0-10)'),
    (fuel_eff, 'Fuel Efficiency (km/l)'),
    (mileage, 'Mileage (in thousands)'),
    (history, 'Ownership History (Hand)'),
    (comfort, 'Practicality & Comfort (Scale 0-10)'),
    (condition, 'Car Condition (Score 0-100)'),
    (safety, 'Safety Rating (Stars)'),
    (suitability, 'Output: Purchase Suitability')
]

# Loop through all variables and create a custom professional plot for each
for var, title in all_vars:
    fig, ax = plt.subplots(figsize=(10, 4))

    for label, term in var.terms.items():
        x = var.universe
        y = term.mf
        line, = ax.plot(x, y, linewidth=2)
        color = line.get_color()

        # Find the peak (highest point) of the current MF
        max_y = np.max(y)
        # Find the indices where the peak occurs (handles flat tops in trapmf)
        peak_indices = np.where(y == max_y)[0]
        # Get the exact center of the peak
        center_peak_idx = peak_indices[len(peak_indices) // 2]
        peak_x = x[center_peak_idx]

        # Clean the label for display (replace underscores with spaces)
        clean_label = label.replace('_', ' ')

        # Add the text directly above the peak
        ax.text(peak_x, max_y + 0.05, clean_label, color=color,
                ha='center', va='bottom', fontsize=11, fontweight='bold')

    ax.set_title(title, fontsize=14, fontweight='bold', pad=15)

```

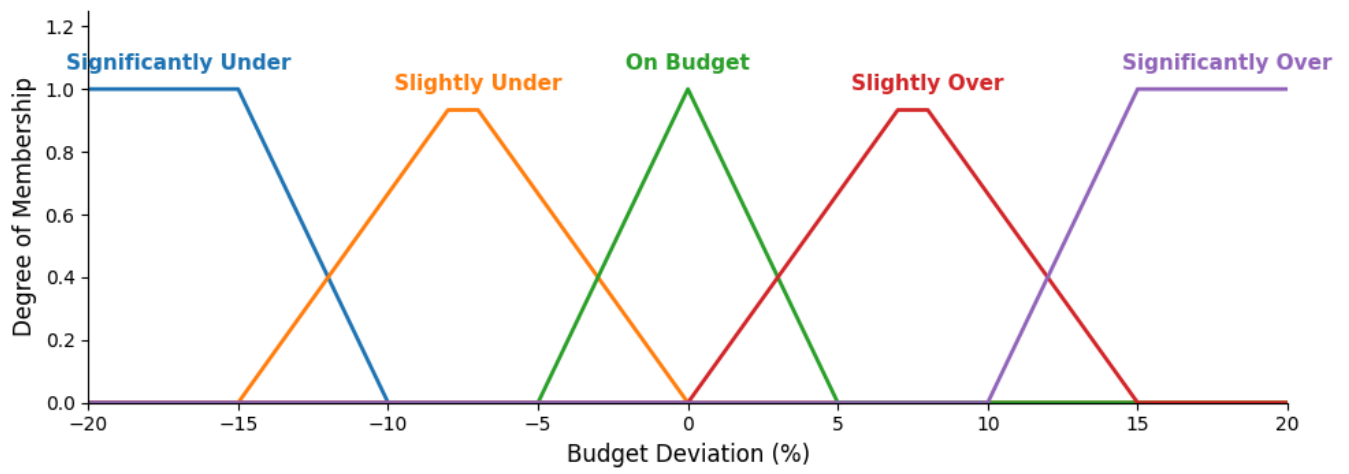
```
ax.set_ylim(0, 1.25) # Give extra vertical space for the text labels
ax.set_xlim(var.universe.min(), var.universe.max())
ax.set_ylabel('Degree of Membership', fontsize=12)
ax.set_xlabel(title, fontsize=12)

# Hide the top and right borders for a cleaner, modern look
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

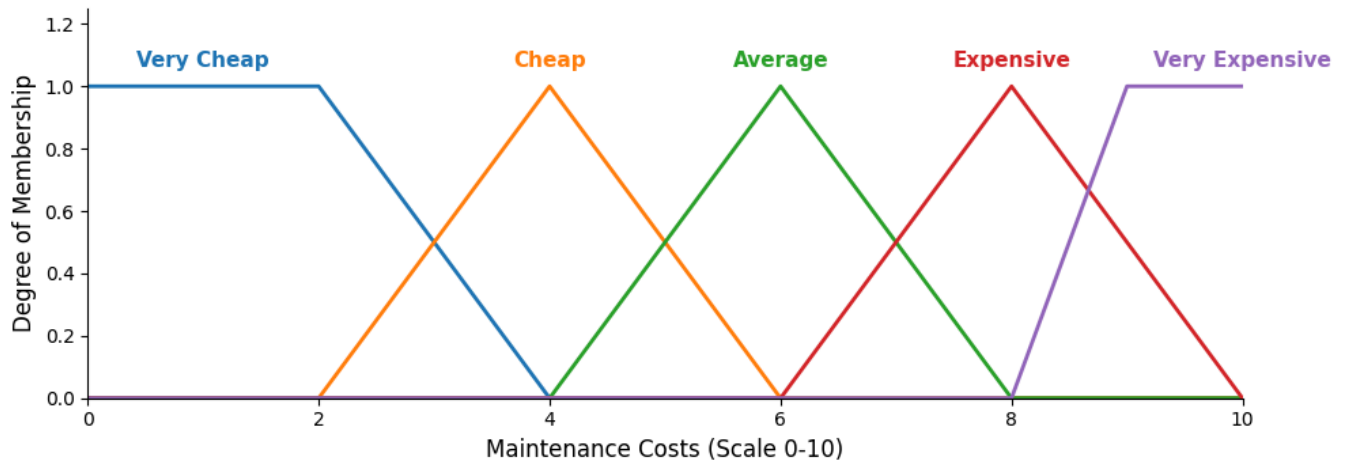
plt.tight_layout()
plt.show()

print("Cell 2 Executed: All customized graphs plotted successfully.")
```

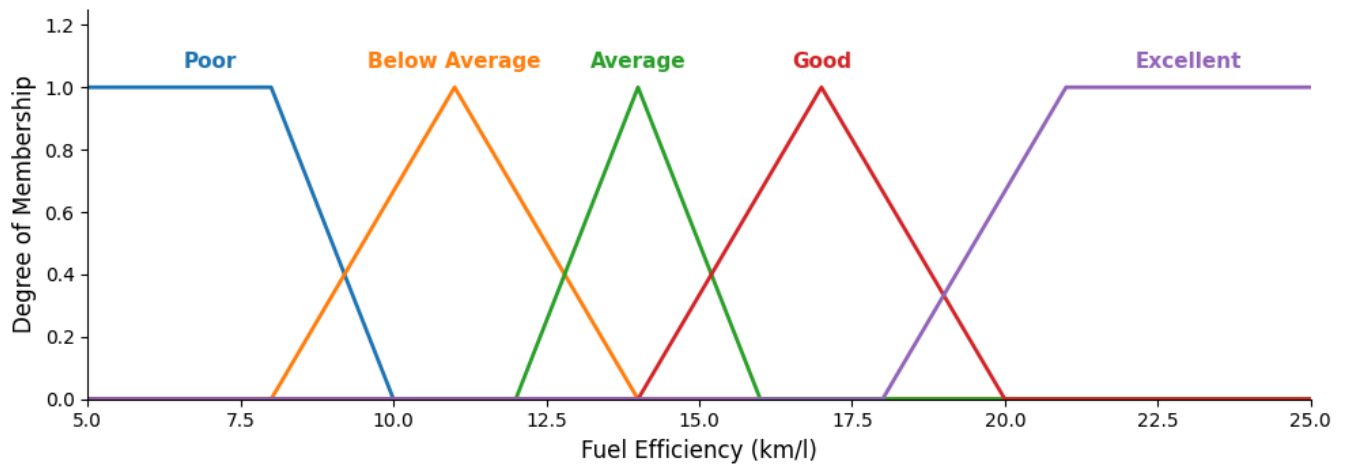
Budget Deviation (%)



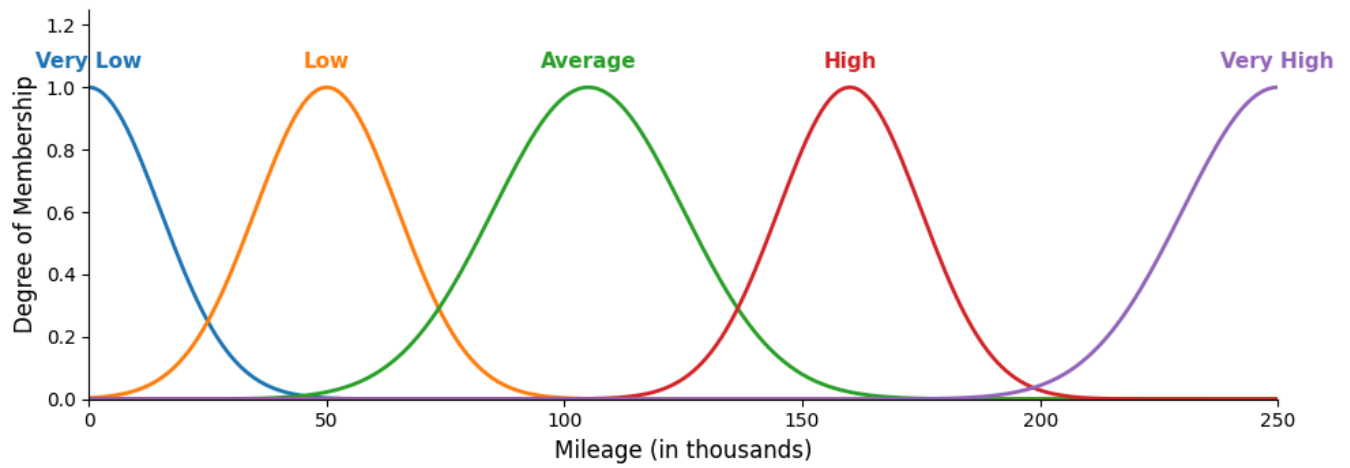
Maintenance Costs (Scale 0-10)



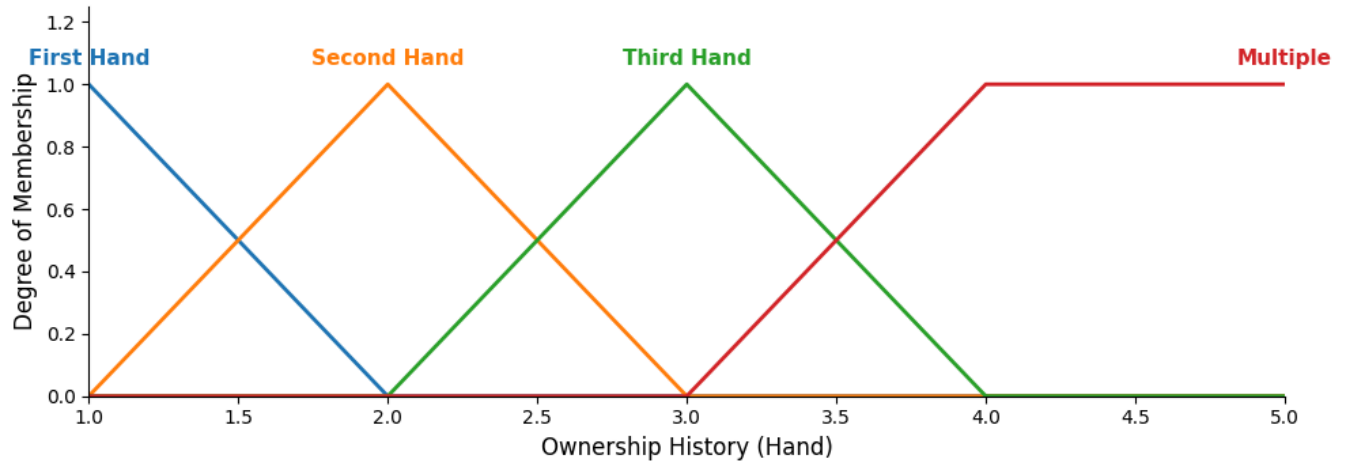
Fuel Efficiency (km/l)



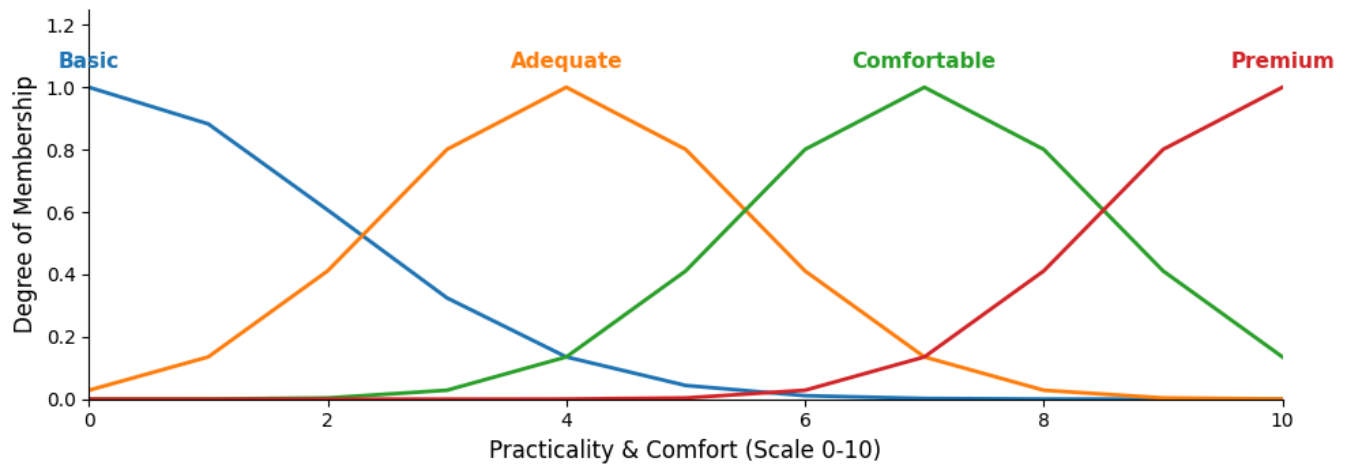
Mileage (in thousands)

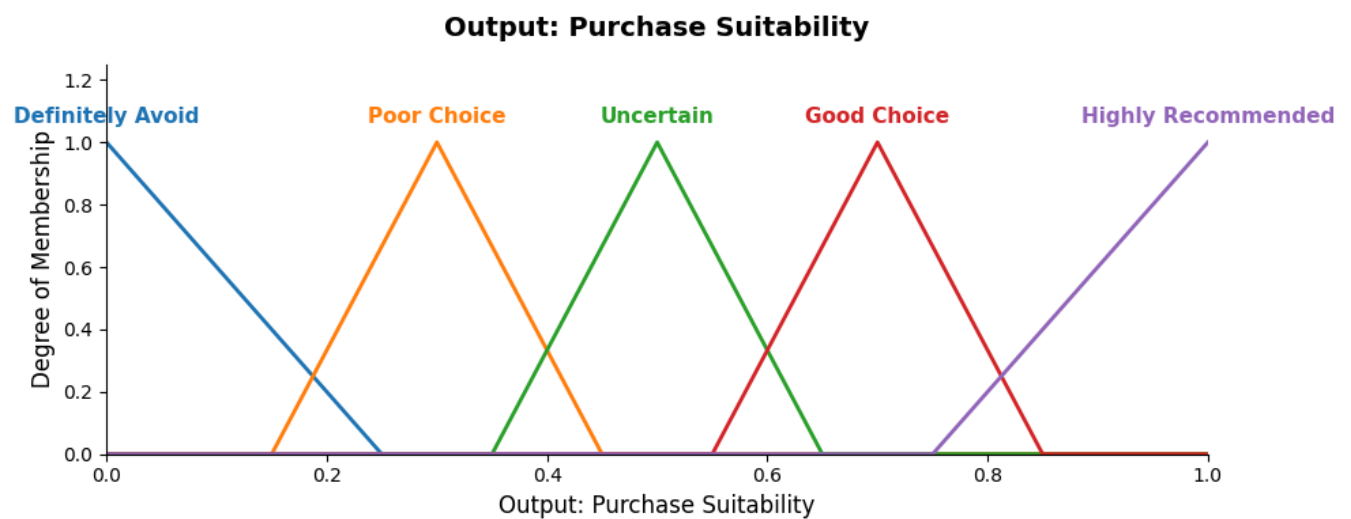
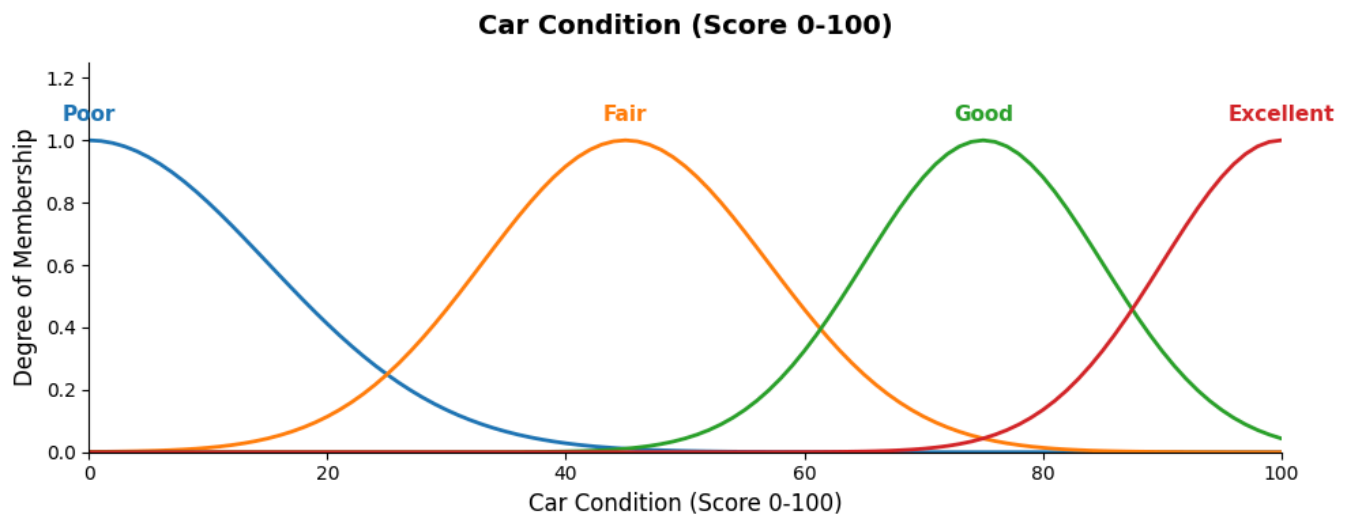


Ownership History (Hand)



Practicality & Comfort (Scale 0-10)





Cell 2 Executed: All customized graphs plotted successfully.

Step 3: The Knowledge Base (Fuzzy IF-THEN Rules)

This cell contains the "brain" of the system. We translate the 20 fuzzy logic rules formulated in our design document into operational code (using operators corresponding to AND and OR), effectively representing the judgment and trade-offs of an automotive expert.

```

In [5]: # Veto Rules
rule1 = ctrl.Rule(safety['Unsafe'], suitability['Definitely_Avoid'])
rule2 = ctrl.Rule(condition['Poor'], suitability['Definitely_Avoid'])
rule3 = ctrl.Rule(budget_dev['Significantly_Over'], suitability['Definitely_Avoid'])
rule4 = ctrl.Rule(mileage['Very_High'] & history['Multiple'], suitability['Definitely_Avoid'])
rule5 = ctrl.Rule(maint_cost['Very_Expensive'] & fuel_eff['Poor'], suitability['Definitely_Avoid'])

# Highly Recommended
rule6 = ctrl.Rule(budget_dev['On_Budget'] & condition['Excellent'] & mileage['Very_Low'] & history['First_Hand'], suitability['Highly_Recommended'])
rule7 = ctrl.Rule(budget_dev['Significantly_Under'] & condition['Excellent'] & safety['Excellent'], suitability['Highly_Recommended'])
rule8 = ctrl.Rule(condition['Excellent'] & maint_cost['Very_Cheap'] & fuel_eff['Excellent'], suitability['Highly_Recommended'])

# Good Choice
rule9 = ctrl.Rule(budget_dev['Slightly_Under'] & condition['Good'] & safety['Good'], suitability['Good_Choice'])
rule10 = ctrl.Rule(budget_dev['On_Budget'] & comfort['Comfortable'] & safety['Good'], suitability['Good_Choice'])
rule11 = ctrl.Rule(mileage['Low'] & history['Second_Hand'] & fuel_eff['Good'], suitability['Good_Choice'])
rule12 = ctrl.Rule(budget_dev['Slightly_Over'] & condition['Excellent'] & maint_cost['Cheap'], suitability['Good_Choice'])
rule13 = ctrl.Rule(budget_dev['Significantly_Under'] & maint_cost['Very_Cheap'] & condition['Fair'], suitability['Good_Choice'])
rule14 = ctrl.Rule(comfort['Premium'] & condition['Good'] & maint_cost['Average'], suitability['Good_Choice'])

# Uncertain
rule15 = ctrl.Rule(budget_dev['On_Budget'] & condition['Fair'] & mileage['Average'], suitability['Uncertain'])
rule16 = ctrl.Rule(fuel_eff['Average'] & maint_cost['Average'] & safety['Acceptable'], suitability['Uncertain'])
rule17 = ctrl.Rule(history['First_Hand'] & mileage['High'] & budget_dev['Slightly_Under'], suitability['Uncertain'])

# Poor Choice
rule18 = ctrl.Rule(budget_dev['Significantly_Under'] & comfort['Premium'] & maint_cost['Very_Expensive'] & mileage['High'], suitability['Poor_Choice'])
rule19 = ctrl.Rule(mileage['High'] & history['Multiple'] & maint_cost['Expensive'], suitability['Poor_Choice'])
rule20 = ctrl.Rule(mileage['High'] & safety['Poor'], suitability['Poor_Choice'])
rule21 = ctrl.Rule(comfort['Basic'] & fuel_eff['Below_Average'], suitability['Poor_Choice'])

print("Cell 3 Executed: 21 Upgraded Expert Rules compiled successfully.")

```

Cell 3 Executed: 21 Upgraded Expert Rules compiled successfully.

Step 4: Inference Engine, Defuzzification & Validation

In the final step, we bundle the rules into a Control System and create a simulator. We input specific car data for our test cases, compute the final score using the Center of Gravity (Centroid) defuzzification method, and visualize the output area and final crisp result.

```

In [6]: # CELL 4: Automated Execution & Validation for All 5 Test Cases
import matplotlib.pyplot as plt
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# =====
# 1. Build Control System & Simulation Engine
# =====
purchase_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6, rule7,
                                   rule8, rule9, rule10, rule11, rule12, rule13,
                                   rule14, rule15, rule16, rule17, rule18, rule19, rule20, rule21])
car_evaluator = ctrl.ControlSystemSimulation(purchase_ctrl)

# =====
# 2. Define All 5 Test Cases from the Table
# =====
test_cases = [
    {
        'model': 'Mazda CX-5 Premium (2020)',
        'inputs': {'Budget_Deviation': -1, 'Maintenance_Costs': 5, 'Fuel_Efficiency': 15.6, 'Mileage': 147, 'Ownership_History': 2, 'Practicality_Comfort': 8, 'Car_Condition': 75, 'Safety_Rating': 5.0}
    },
    {
        'model': 'Hyundai Sonata Hybrid (2021)',
        'inputs': {'Budget_Deviation': 25, 'Maintenance_Costs': 4, 'Fuel_Efficiency': 20.0, 'Mileage': 130, 'Ownership_History': 2, 'Practicality_Comfort': 9, 'Car_Condition': 85, 'Safety_Rating': 5.0}
    },
    {
        'model': 'Skoda Karoq Style (2020)',
        'inputs': {'Budget_Deviation': -13, 'Maintenance_Costs': 5, 'Fuel_Efficiency': 18.2, 'Mileage': 85, 'Ownership_History': 2, 'Practicality_Comfort': 7, 'Car_Condition': 85, 'Safety_Rating': 5.0}
    },
    {
        'model': 'BMW X4 Sport (2017)',
        'inputs': {'Budget_Deviation': 9, 'Maintenance_Costs': 8, 'Fuel_Efficiency': 13.7, 'Mileage': 96, 'Ownership_History': 2, 'Practicality_Comfort': 8, 'Car_Condition': 70, 'Safety_Rating': 4.5}
    },
    {
        'model': 'Porsche Cayenne Luxury (2012)',
        'inputs': {'Budget_Deviation': -15, 'Maintenance_Costs': 10, 'Fuel_Efficiency': 10.1, 'Mileage': 166, 'Ownership_History': 2, 'Practicality_Comfort': 9, 'Car_Condition': 65, 'Safety_Rating': 4.0}
    }
]

var_list = [budget_dev, maint_cost, fuel_eff, mileage, history, comfort, condition, safety]

rules_viz = [
    {'vars': {'Safety_Rating': 'Unsafe'}, 'op': 'AND', 'out': 'Definitely_Avoid'},
    {'vars': {'Car_Condition': 'Poor'}, 'op': 'AND', 'out': 'Definitely_Avoid'},

```

```

        {'vars': {'Budget_Deviation': 'Significantly_Over'}, 'op': 'AND', 'out': 'Definitely_Avoid'},
        {'vars': {'Mileage': 'Very_High', 'Ownership_History': 'Multiple'}, 'op': 'AND', 'out': 'Definitely_Avoid'},
        {'vars': {'Maintenance_Costs': 'Very_Expensive', 'Fuel_Efficiency': 'Poor'}, 'op': 'AND', 'out': 'Definitely_Avoid'},
        {'vars': {'Budget_Deviation': 'On_Budget', 'Car_Condition': 'Excellent', 'Mileage': 'Very_Low', 'Ownership_History': 'First_Hand'}, 'op': 'AND', 'out': 'Highly_Recommended'},
        {'vars': {'Budget_Deviation': 'Significantly_Under', 'Car_Condition': 'Excellent', 'Safety_Rating': 'Excellent'}, 'op': 'AND', 'out': 'Highly_Recommended'},
        {'vars': {'Car_Condition': 'Excellent', 'Maintenance_Costs': 'Very_Cheap', 'Fuel_Efficiency': 'Excellent'}, 'op': 'AND', 'out': 'Highly_Recommended'},
        {'vars': {'Budget_Deviation': 'Slightly_Under', 'Car_Condition': 'Good', 'Safety_Rating': 'Good'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Budget_Deviation': 'On_Budget', 'Practicality_Comfort': 'Comfortable', 'Safety_Rating': 'Good'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Mileage': 'Low', 'Ownership_History': 'Second_Hand', 'Fuel_Efficiency': 'Good'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Budget_Deviation': 'Slightly_Over', 'Car_Condition': 'Excellent', 'Maintenance_Costs': 'Cheap'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Budget_Deviation': 'Significantly_Under', 'Maintenance_Costs': 'Very_Cheap', 'Car_Condition': 'Fair'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Practicality_Comfort': 'Premium', 'Car_Condition': 'Good', 'Maintenance_Costs': 'Average'}, 'op': 'AND', 'out': 'Good_Choice'},
        {'vars': {'Budget_Deviation': 'On_Budget', 'Car_Condition': 'Fair', 'Mileage': 'Average'}, 'op': 'AND', 'out': 'Uncertain'},
        {'vars': {'Fuel_Efficiency': 'Average', 'Maintenance_Costs': 'Average', 'Safety_Rating': 'Acceptable'}, 'op': 'AND', 'out': 'Uncertain'},
        {'vars': {'Ownership_History': 'First_Hand', 'Mileage': 'High', 'Budget_Deviation': 'Slightly_Under'}, 'op': 'AND', 'out': 'Uncertain'},
        {'vars': {'Budget_Deviation': 'Significantly_Under', 'Practicality_Comfort': 'Premium', 'Maintenance_Costs': 'Very_Expensive', 'Mileage': 'High'}, 'op': 'AND', 'out': 'Poor_Choice'},
        {'vars': {'Mileage': 'High', 'Ownership_History': 'Multiple', 'Maintenance_Costs': 'Expensive'}, 'op': 'AND', 'out': 'Poor_Choice'},
        {'vars': {'Mileage': 'High', 'Safety_Rating': 'Poor'}, 'op': 'AND', 'out': 'Poor_Choice'},
        {'vars': {'Practicality_Comfort': 'Basic', 'Fuel_Efficiency': 'Below_Average'}, 'op': 'AND', 'out': 'Poor_Choice'}
    ]

# =====
# 3. Loop Execution for All Test Cases
# =====
for idx, tc in enumerate(test_cases):
    test_inputs = tc['inputs']
    var_names = list(test_inputs.keys())

    print("\n" + "="*65)
    print(f"  TEST CASE {idx+1}: {tc['model']}")
    print("="*65)
    for key, val in test_inputs.items():
        print(f"    * {key.replace('_', ' '):<25}: {val}")
    print("="*65)

# Compute Fuzzy Logic Output
for key, val in test_inputs.items():
    car_evaluator.input[key] = val
car_evaluator.compute()
final_score = car_evaluator.output['Purchase_Suitability']

```

```

# Determine linguistic category
best_category = ""
max_mem = -1
for term_name, term in suitability.terms.items():
    mem = fuzz.interp_membership(suitability.universe, term.mf, final_score)
    if mem > max_mem:
        max_mem = mem
        best_category = term_name.replace('_', ' ')

print(f"✓ Final Calculated Centroid Score : {final_score:.3f}")
print(f"⏏ System Category Verdict : {best_category}")
print("="*65)

# -----
# Fast Mathematical Calculation of Aggregated Area (Replaces the broken internal call)
# -----
aggregated_out = np.zeros_like(suitability.universe)

for rule in rules_viz:
    activations = []
    for v_name, t_name in rule['vars'].items():
        v_idx = var_names.index(v_name)
        membership = fuzz.interp_membership(var_list[v_idx].universe, var_list[v_idx][t_name].mf, test_inputs[v_name])
        activations.append(membership)

    if rule['op'] == 'OR':
        rule_fire = max(activations) if activations else 0
    else:
        rule_fire = min(activations) if activations else 0

    out_mf = suitability[rule['out']].mf
    truncated = np.fmin(rule_fire, out_mf)
    aggregated_out = np.fmax(aggregated_out, truncated)

# -----
# Final Defuzzification Visualization
# -----
fig_final, ax_final = plt.subplots(figsize=(8, 3.5))

for term in suitability.terms.values():
    ax_final.plot(suitability.universe, term.mf, color='#dddddd', linestyle='--', linewidth=1)

    ax_final.fill_between(suitability.universe, 0, aggregated_out, color='#8e44ad', alpha=0.5)
    ax_final.plot(suitability.universe, aggregated_out, color='#2c3e50', linewidth=2)

# --- NEW: Calculate the visual center of gravity for the dot ---
# Find the height (Y) of the purple area exactly at our Centroid (X)
y_at_centroid = np.interp(final_score, suitability.universe, aggregated_out)
y_center = y_at_centroid / 2 # The vertical middle of the area

# Draw the dashed vertical drop-line
ax_final.vlines(x=final_score, ymin=0, ymax=y_at_centroid, color='#e74c3c', linestyle='--', linewidth=2, zorder=4)

```

```

# Plot the dot floating inside the area
ax_final.plot(final_score, y_center, marker='o', color='#e74c3c', markersize=10, zorder=5)

# Place the text slightly above the top of the line
ax_final.text(final_score, y_at_centroid + 0.05, f'Centroid\n{final_score:.3f}', color='#e74c3c',
              ha='center', va='bottom', fontweight='bold', fontsize=12)

ax_final.set_title(f'{tc["model"]}\nVerdict: {best_category}', fontsize=12, fontweight='bold', pad=10)
ax_final.set_xlim(0, 1)
ax_final.set_ylim(0, 1.15) # Slightly increased to give room for text
ax_final.spines['top'].set_visible(False)
ax_final.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()
plt.close()

```

=====

▯ TEST CASE 1: Mazda CX-5 Premium (2020)

=====

* Budget Deviation : -1
* Maintenance Costs : 5
* Fuel Efficiency : 15.6
* Mileage : 147
* Ownership History : 2
* Practicality Comfort : 8
* Car Condition : 75
* Safety Rating : 5.0

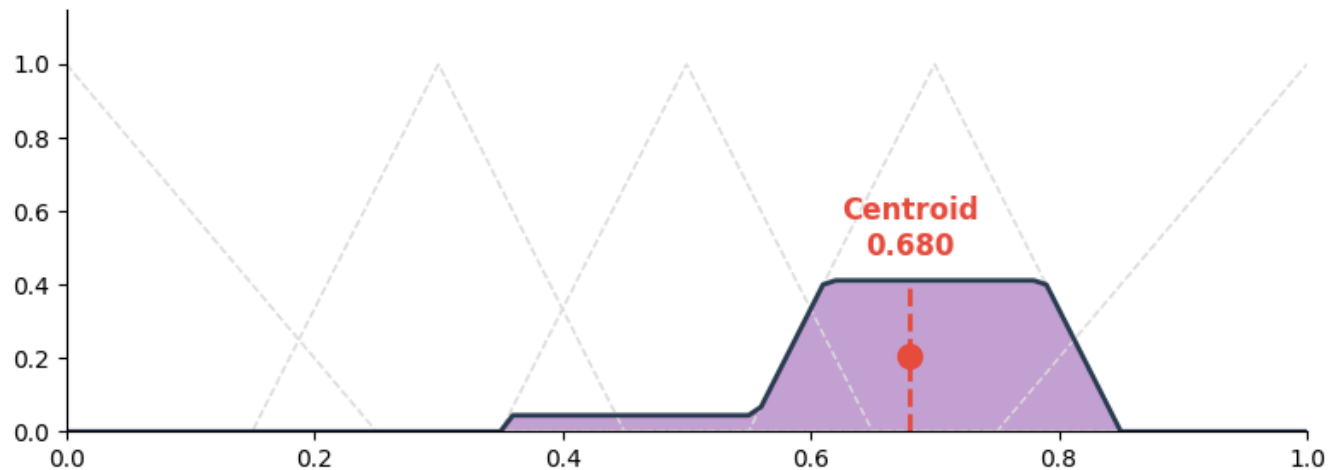
=====

✓ Final Calculated Centroid Score : 0.680

▯ System Category Verdict : Good Choice

=====

Mazda CX-5 Premium (2020)
Verdict: Good Choice



▯ TEST CASE 2: Hyundai Sonata Hybrid (2021)

=====

* Budget Deviation : 25
* Maintenance Costs : 4
* Fuel Efficiency : 20.0
* Mileage : 130
* Ownership History : 2
* Practicality Comfort : 9
* Car Condition : 85
* Safety Rating : 5.0

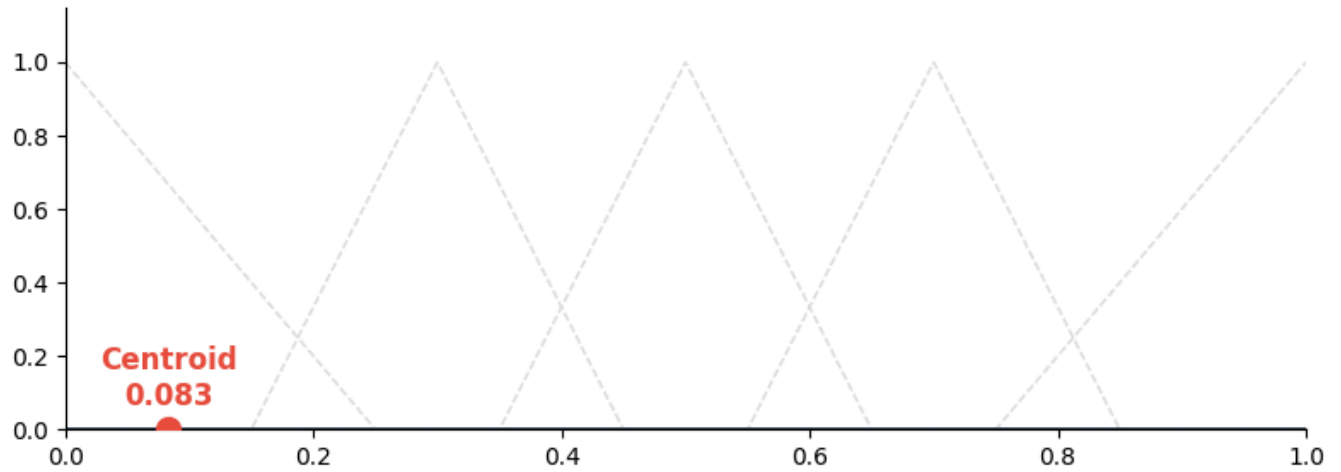
=====

✓ Final Calculated Centroid Score : 0.083

▯ System Category Verdict : Definitely Avoid

=====

Hyundai Sonata Hybrid (2021) Verdict: Definitely Avoid



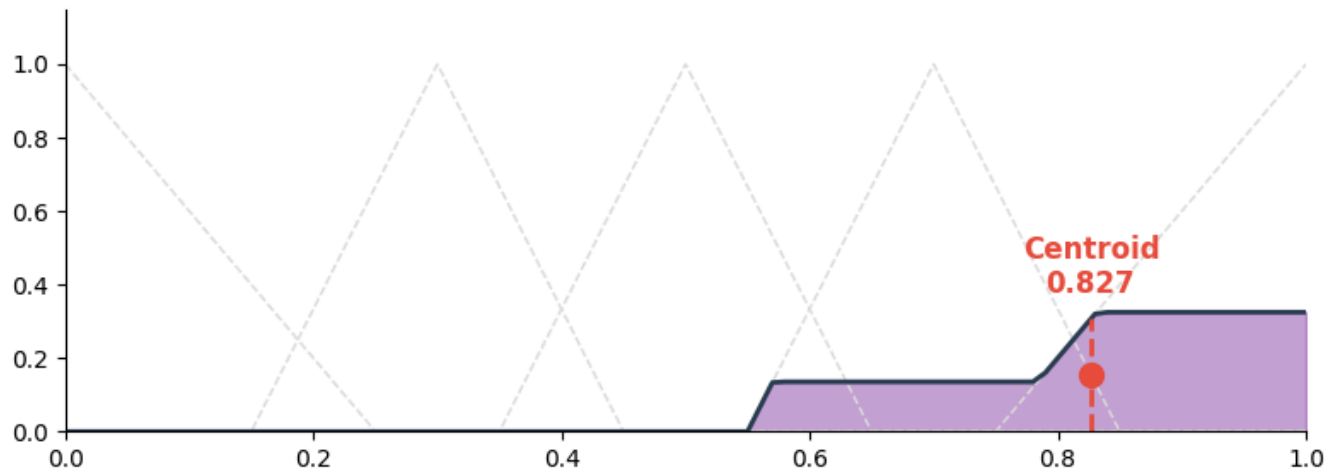
TEST CASE 3: Skoda Karoq Style (2020)

* Budget Deviation : -13
* Maintenance Costs : 5
* Fuel Efficiency : 18.2
* Mileage : 85
* Ownership History : 2
* Practicality Comfort : 7
* Car Condition : 85
* Safety Rating : 5.0

✓ Final Calculated Centroid Score : 0.827

□ System Category Verdict : Highly Recommended

Skoda Karoq Style (2020) Verdict: Highly Recommended



=====

□ TEST CASE 4: BMW X4 Sport (2017)

=====

* Budget Deviation : 9
* Maintenance Costs : 8
* Fuel Efficiency : 13.7
* Mileage : 96
* Ownership History : 2
* Practicality Comfort : 8
* Car Condition : 70
* Safety Rating : 4.5

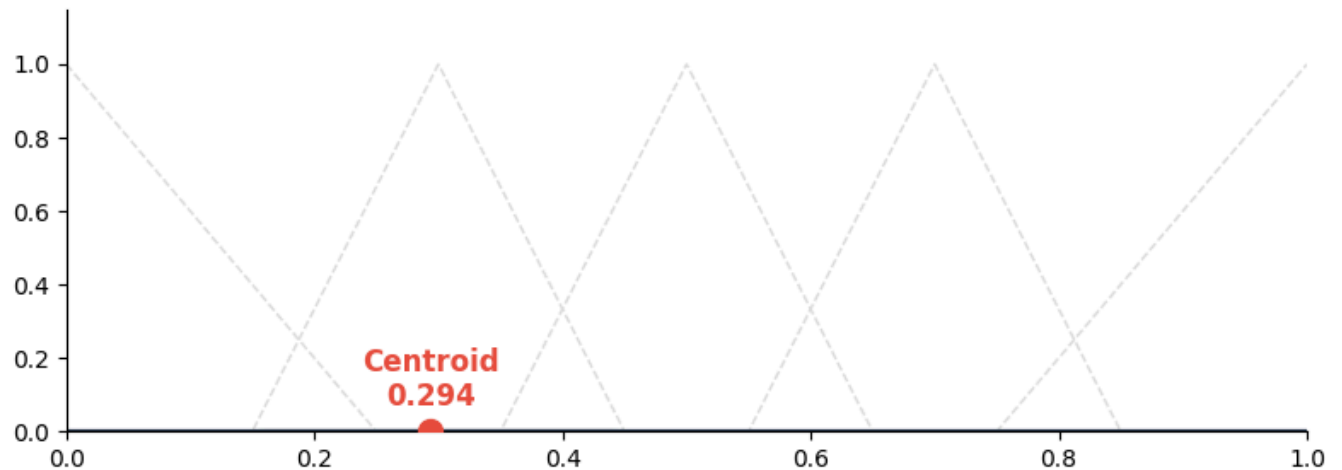
=====

✓ Final Calculated Centroid Score : 0.294

□ System Category Verdict : Poor Choice

=====

BMW X4 Sport (2017)
Verdict: Poor Choice



=====

□ TEST CASE 5: Porsche Cayenne Luxury (2012)

=====

* Budget Deviation : -15
* Maintenance Costs : 10
* Fuel Efficiency : 10.1
* Mileage : 166
* Ownership History : 2
* Practicality Comfort : 9
* Car Condition : 65
* Safety Rating : 4.0

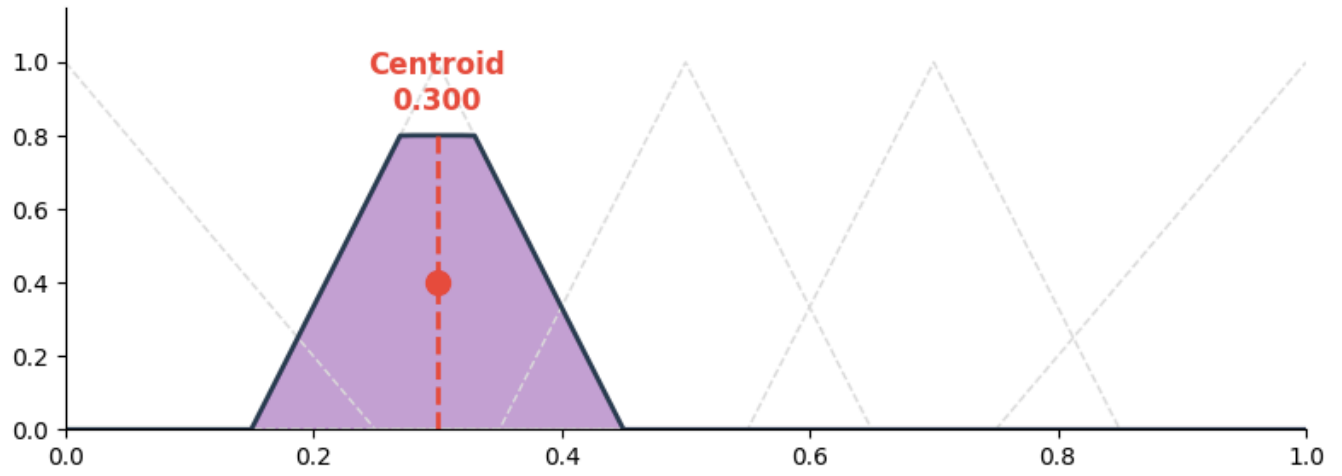
=====

✓ Final Calculated Centroid Score : 0.300

□ System Category Verdict : Poor Choice

=====

Porsche Cayenne Luxury (2012)
Verdict: Poor Choice



```

In [7]: # CELL 5: Full Master Rule Viewer Grids for ALL 5 Test Cases
# (Relies on data, rules, and variables already defined in Cell 4)

import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz

# Loop through all 5 test cases defined in Cell 4
for idx, tc in enumerate(test_cases):
    test_inputs = tc['inputs']
    var_names = list(test_inputs.keys())

    print(f"\nGenerating Master Grid for: {tc['model']}...")

    # Create the massive figure for the current car
    fig_grid, axes = plt.subplots(nrows=21, ncols=9, figsize=(18, 27), facecolor='#ffffff')
    fig_grid.subplots_adjust(wspace=0.15, hspace=0.3)

    for row, rule in enumerate(rules_viz):
        activations = []
        for v_name, t_name in rule['vars'].items():
            v_idx = var_names.index(v_name)
            membership = fuzz.interp_membership(var_list[v_idx].universe, var_list[v_idx][t_name].mf, test_inputs[v_name])
            activations.append(membership)

        rule_fire = max(activations) if rule['op'] == 'OR' and activations else (min(activations) if activations else 0)

        for col in range(8):
            ax = axes[row, col]
            ax.set_xticks([]); ax.set_yticks([])
            for spine in ax.spines.values(): spine.set_color('#dddddd')

            v_name = var_names[col]
            universe = var_list[col].universe
            val = test_inputs[v_name]

            for term in var_list[col].terms.values():
                ax.plot(universe, term.mf, color='#eeeeee', linewidth=1)
                ax.axvline(x=val, color='#ff4d4d', linewidth=1.5, zorder=3)

            if v_name in rule['vars']:
                t_name = rule['vars'][v_name]
                mf = var_list[col][t_name].mf
                membership = fuzz.interp_membership(universe, mf, val)
                ax.plot(universe, mf, color='#0052cc', linewidth=2, zorder=4)
                ax.fill_between(universe, 0, np.fmin(mf, membership), color='#feaa7', alpha=0.9, zorder=2)

            ax.set_ylim(0, 1.05)
            if row == 0: ax.set_title(v_name.split('_')[0], fontsize=10, fontweight='bold', color='#333')

            ax_out = axes[row, 8]
            ax_out.set_xticks([]); ax_out.set_yticks([])
            for spine in ax_out.spines.values(): spine.set_color('#dddddd')

            out_universe = suitability.universe
            for term in suitability.terms.values():

```

```

        ax_out.plot(out_universe, term.mf, color='#eeeeee', linewidth=1)

        out_mf = suitability[rule['out']].mf
        if rule_fire > 0:
            ax_out.plot(out_universe, out_mf, color='#0052cc', linewidth=2, zorder=4)
            ax_out.fill_between(out_universe, 0, np.fmin(rule_fire, out_mf), color='#0052cc', alpha=0.5, zorder=2)
        else:
            ax_out.plot(out_universe, out_mf, color='#0052cc', linewidth=1, linestyle='--', alpha=0.5)

        ax_out.set_ylim(0, 1.05)
        if row == 0: ax_out.set_title('Output', fontsize=10, fontweight='bold', color='#333')
        axes[row, 0].text(-0.2, 0.5, f'R{row+1}', transform=axes[row, 0].transAxes, fontsize=10, fontweight='bold', va='center', ha='right', color='#555')

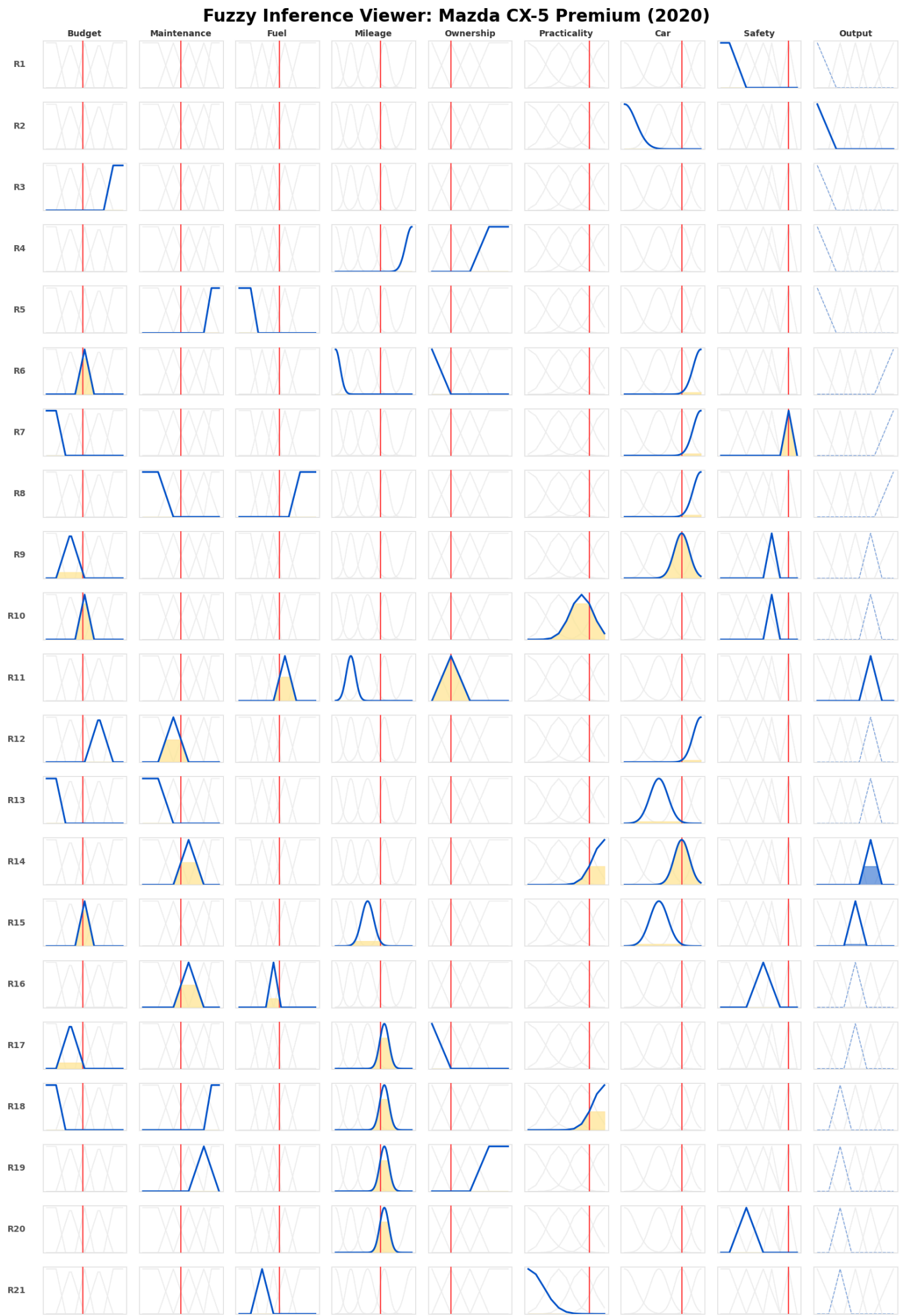
        fig_grid.suptitle(f"Fuzzy Inference Viewer: {tc['model']}", fontsize=20, fontweight='bold', y=0.90)

        # Display the grid for this specific car
        plt.show()

        # CRITICAL: Close the figure immediately after showing it to prevent Jupyter from crashing!
        plt.close(fig_grid)

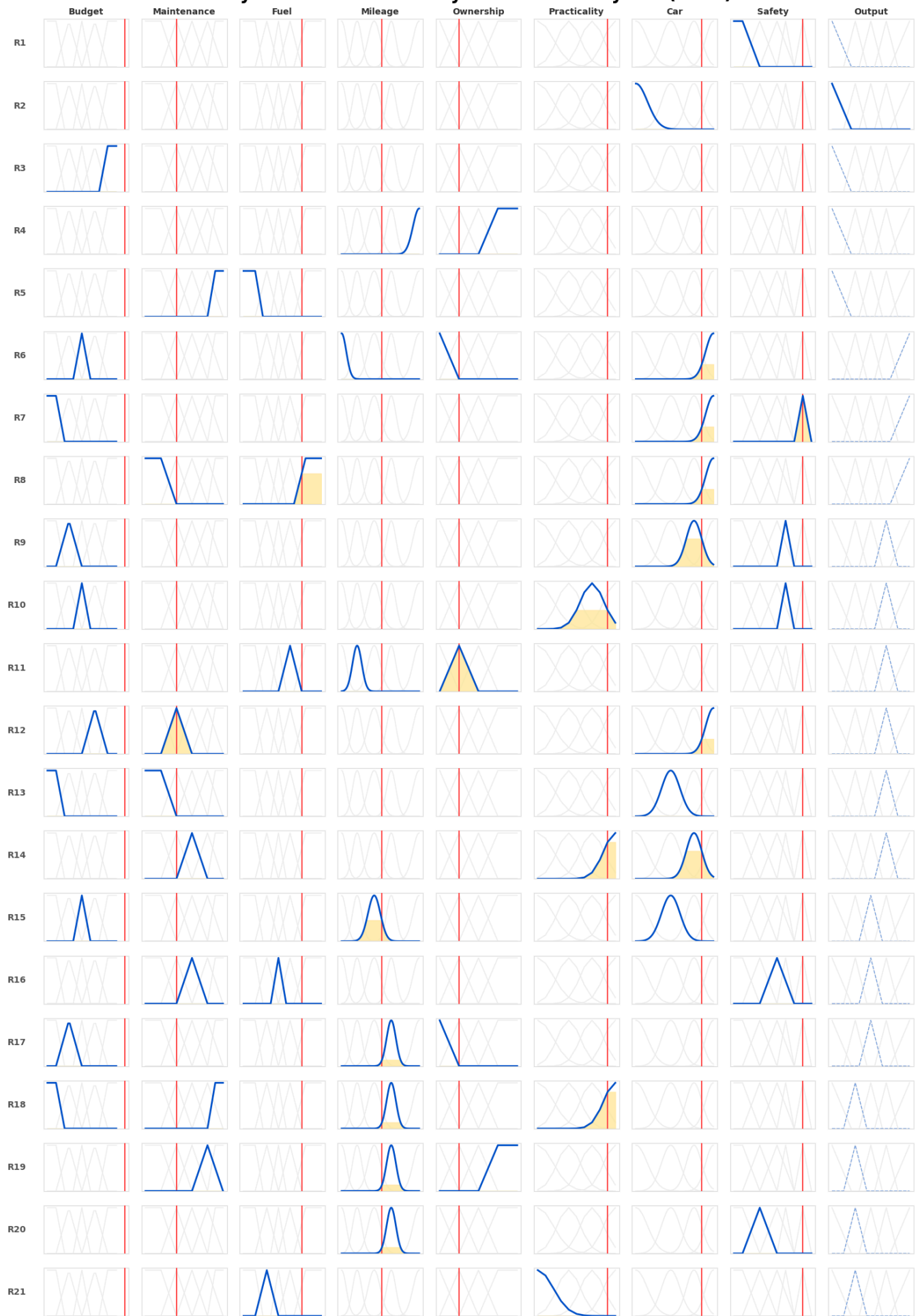
```

Generating Master Grid for: Mazda CX-5 Premium (2020)...



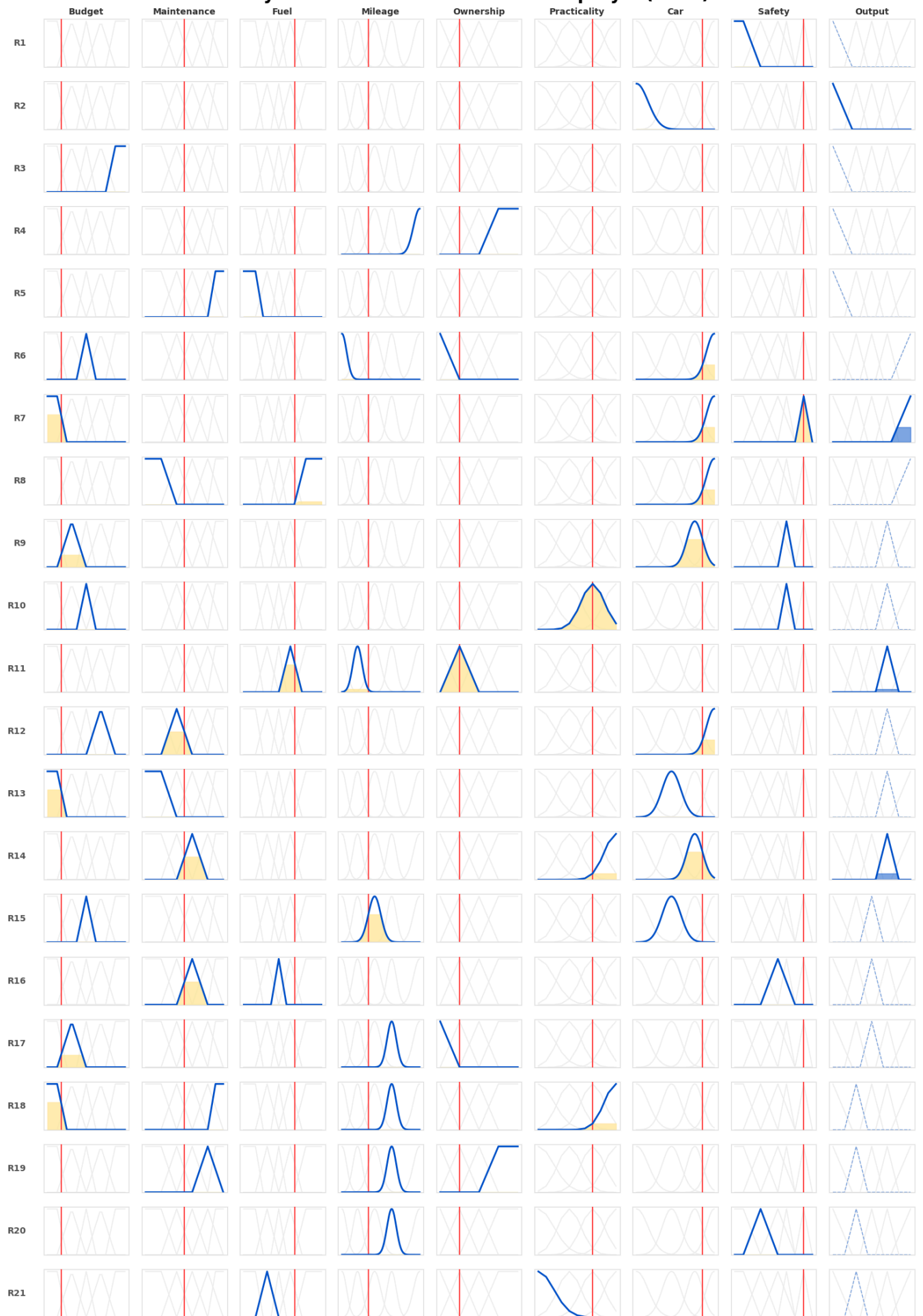
Generating Master Grid for: Hyundai Sonata Hybrid (2021)...

Fuzzy Inference Viewer: Hyundai Sonata Hybrid (2021)



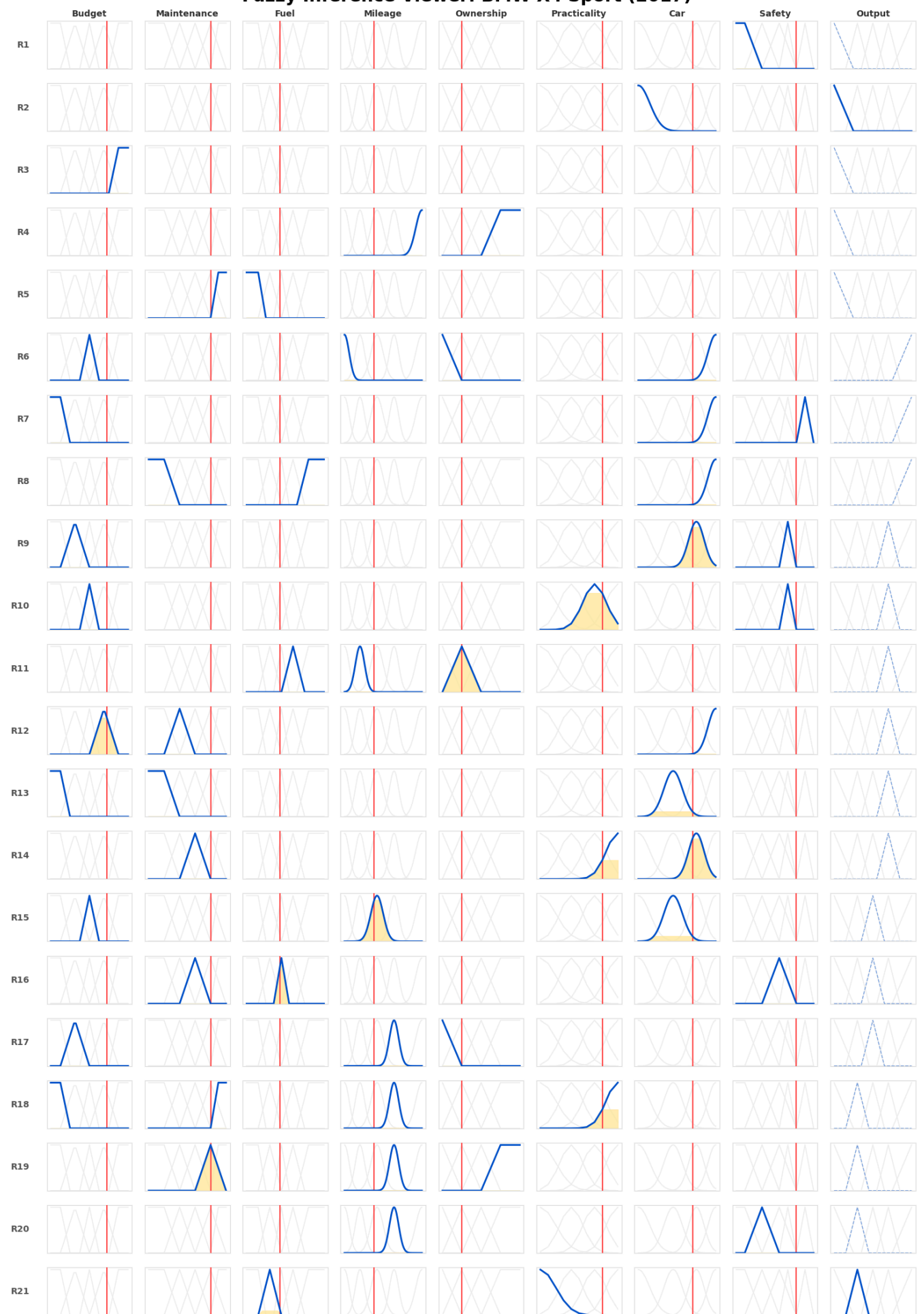
Generating Master Grid for: Skoda Karoq Style (2020)...

Fuzzy Inference Viewer: Skoda Karoq Style (2020)



Generating Master Grid for: BMW X4 Sport (2017)...

Fuzzy Inference Viewer: BMW X4 Sport (2017)



Generating Master Grid for: Porsche Cayenne Luxury (2012)...

Fuzzy Inference Viewer: Porsche Cayenne Luxury (2012)

